Siemens Simatic PLCs -- Blocks

- Siemens PLCs are programmed via a software package called **TIA Portal**
- A PLC program typically comprises **program blocks** that are called **over the code flow**
  - In case of Siemens PLCs, such blocks are
    - **Functions, FC**
    - **Function Blocks, FB**
- Other blocks are made available by the operating system
  - In case of Siemens PLCs these blocks are called **Organization Blocks, OBs**
- Further blocks are used to store data
  - In case of Siemens PLCs, these are called **Data Blocks, DB**

Organization Blocks

- Made available by the operating system
- Identified as **OBn**, where n is the number of the Organisation Block
- Can **not be called/invoked by the user program** (they are put in execution by the operating system)
- They **include code** written by the user
- **OB1 is the main block**, put in execution at the **beginning** of each PLC cycle
- The end of OB1 represents the **end of the current PLC cycle**
- Typical employments of OBs are
  - **Interrupt handling**
  - **Handling of system/program errors**
  - **Setting of the maximum PLC cycle time**

Functions and Function Blocks

- Equivalent to **routines** of common programming languages
- Numbered from 1 to 256 -512, etc., depending on the PLC type
- They are **called/invoked by the user program**
- The calling instruction is **CALL.** Examples**:** CALL FC12, CALL FC99, CALL FB102
- Functions and Function Blocks **may be parametrized**
- They are written using formal parameters
- When they are invoked, **formal parameters are replaced by real operands**
- Example: FC100 has been written to acquire two temperature signals. It has two formal parameters, namely **TEMP1 and TEMP2**. We can use FC100 to acquire the two temperature signals reported by the **analogue inputs input_a and input_b.**
- The calling syntax is

  CALL FC100
  TEMP1=input_a
  TEMP2=input_b

NOTE: Function Blocks have a different calling syntax, in that they use an instance data block. However, **they will not be used in this course**

Operands

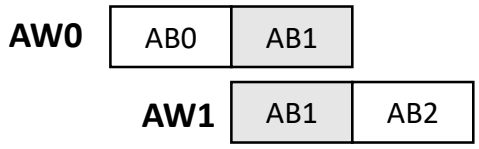
- Siemens PLCs use several types of operands. Those that will be considered in this course are
  - **Inputs**
    - Refer to **physical inputs** of the plant.
    - The operand symbol used in programming is **E** (Eingabe)
  - **Outputs**
    - Refer to **physical outputs** of the plant
    - The operand symbol used in programming is **A** (Ausgabe)
  - **Merkers**
    - Registers available in the system memory, accessible by the user program
    - Used as permanent/working memories
    - The operand symbol used in programming is **M**
  - **Data**
    - Registers available in the user memory, accessible by the user program
    - Used as permanent/working memories
- **NOTE: Inputs and Outputs are plant variables that are mapped on the user program. This requires great attention in programming. For example, the simple set of an output bit by the PLC program may start a high power motor**

Use of Operands

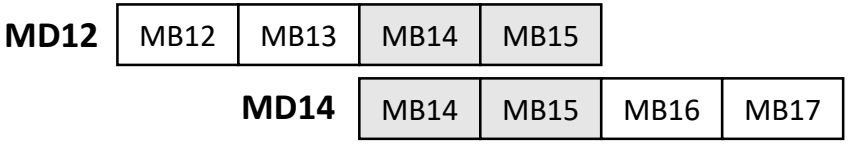
- Operands are stored as **bytes**. However, they can be **grouped** in more complex structures such as **Words and Double Words** and, also, **decoupled in single bits**.
- Thus, in a PLC program, operands may **be referred to as bytes, words, Double Words and bits**
  - **Bytes**
    - **EB0, EB22, EB100** are, respectively, the input bytes #0, #22 and #100
    - **AB1** is the output byte #1. **MB29** is the merker byte #29
  - **Bits**
    - **E123.7** is the bit **#7** of the **input byte #123**
    - **A0.1** is the bit **#1** of the **output byte #0**
    - **M0.0** is the bit #0 of the **merker byte #0**
    - **M12.8** is a **wrong address**
- **NOTE: the instructions used to refer to bytes/words/double words are different from those used to refer to bits**

Words and Double Words

- Words and Double Words are obtained grouping two and four bytes respectively.
  - AW0 is the output word 0. **It comprises the bytes AB0 and AB1**
  - EW22 is the input word 22. **It comprises the bytes EB22 and EB23**
  - MD16 is the double merker word 16. **It comprises MB16, MB17, MB18 and MB19**
- Notably, **bytes, words and double words may overlap.** This may represent a problem



AW0 and AW1 have **AB1 in common**. Thus, writing on AW0 has the effect of **modifying also AW1**



MD12 and MD14 have MB14 and MB16 in common. Thus, writing on these two bytes **has effect on both the double words**

- **Good programming rule:**
  - Use only **even numbers for words** (AW12, MW18, EW100, etc.)
  - Use only **multiple of four numbers for double words** (ED0, MD32, AD16, etc.)